Concurrent Servers: Processes and Threads

Lecture 12

Computer Networks and Distributed Programming

Where we are: three mechanisms, one server

You have a server that works and cannot be used: it serves one client, and everybody else waits for that one to leave.

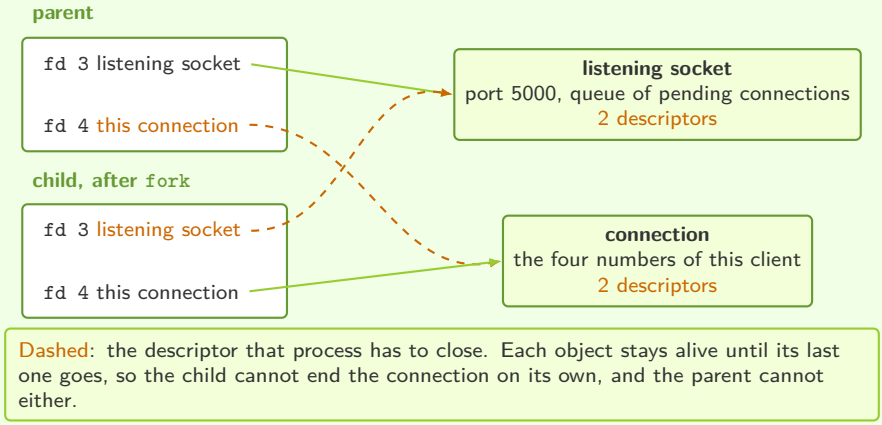
- Since then we took apart the three mechanisms that fix it — a second process, a second thread, and a mutex to make sharing safe.
- None of them ever touched a socket: the `fork` printed a line twice, the threads incremented a counter.
- Today they go into the accept loop, and the same service is written twice: once with processes, once with threads.
- The two versions differ in one thing that then decides everything else — whether the code serving one client can see the others.

What makes a server concurrent

- The kernel already accepts connections while you are busy: they sit in the `listen` queue. What it cannot do is serve them.
- So the queue buys space, not time. A client waiting in it waits exactly as long, and when the queue fills up the next one is refused.
- A server is concurrent when one client's think time is not another client's wait: the second is served while the first says nothing.
- And that is almost all of the time: a chat server spends its life blocked in read, not computing.
- Three ways to get there — one process per client, one thread per client, one flow of control watching many connections. In that order.

One process per client

Nothing new in the mechanism: after `accept` the server forks, and the child inherits both sockets.



The concurrent server: what changes before the loop

```c
static void reap(int sig) {                  /* installed once, before any fork */
    int saved = errno;
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ;                                    /* every child that has ended */
    errno = saved;
}

int main(void) {
    struct sigaction sa;
    int listen_fd = listening_socket(5000);   /* socket, SO_REUSEADDR, bind, listen */

    sa.sa_handler = reap;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGCHLD, &sa, NULL);           /* or the children pile up as zombies */
    signal(SIGPIPE, SIG_IGN);                /* a client that leaves must not kill us */
```

Two lines, and both are about other people ending: a child that died, and a client that closed while we were writing to it.

The accept loop that forks

```
for (;;) {
    int fd = accept(listen_fd, NULL, NULL);
    if (fd < 0) {
        if (errno == EINTR) continue;       /* a child died while we waited */
        perror("accept"); continue;
    }
    pid_t pid = fork();
    if (pid < 0) { perror("fork"); close(fd); continue; }  /* refuse this one only */

    if (pid == 0) {                         /* the child: this connection only */
        close(listen_fd);
        serve(fd);
        close(fd);
        _exit(0);                           /* not exit(): the buffers are not ours */
    }
    close(fd);                              /* the parent is back in accept at once */
}
```

The child: the service did not change

```c
static unsigned long served = 0;          /* how many messages we have relayed */

static void serve(int fd) {                /* one whole conversation */
    char buf[256], head[32];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0) {
        int h = snprintf(head, sizeof head, "[%lu] ", ++served);
        if (write(fd, head, h) != h) break;
        if (write(fd, buf, n) != n) break; /* whatever arrived: not yet a line */
    }
    if (n < 0) perror("read");
}
```

The concurrency is entirely in the loop, not in the service: this function is the sequential server's, unchanged. Keep an eye on `served`.

Twenty clients at once

```
$ ./chatd &
$ for i in $(seq 1 20); do sleep 60 | nc 127.0.0.1 5000 & done
$ ps --ppid $(pgrep -o chatd) -o pid,stat,cmd | head -4
   PID STAT CMD
  7413 S    ./chatd          one child per connected client
  7414 S    ./chatd
  7415 S    ./chatd
$ ss -tn state established '( sport = :5000 )' | wc -l
20
```

Twenty clients, twenty children, and every one of them served. So type a line into one of them: who sees it?

Nobody but the sender

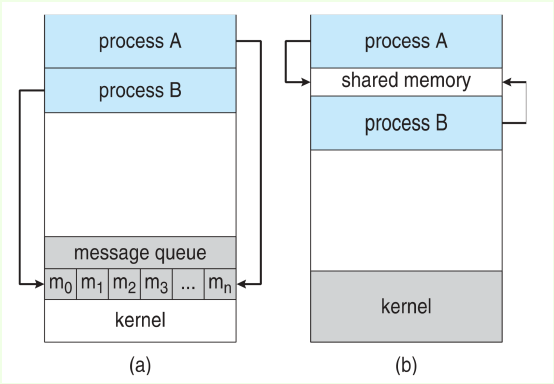
Client 1 sends a line. Nineteen other clients are connected. What do they see?

Nothing, ever. The line was read by one child, and that child has no descriptor to any other connection: the parent closed each one, and the siblings were forked before it existed.

- The same reason makes `served` useless: every client is told it is message number one, because every child got its own copy at `fork` time.
- What we built is twenty independent echo servers that happen to share a port — fine for a web server, useless for a chat.
- Separate address spaces are exactly what we asked for. If the processes have to talk, the kernel has to carry it.

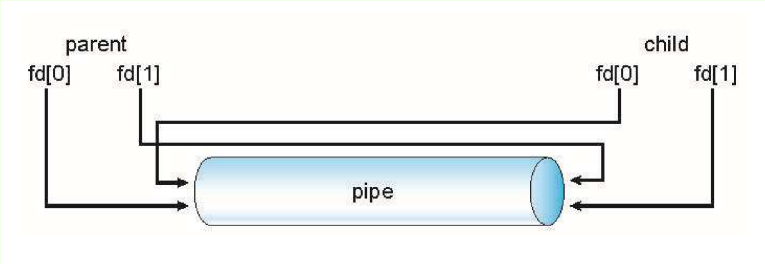
Two ways for processes to talk

- Message passing: every exchange is a system call, the kernel holds the data, and the arrival order is the kernel's guarantee.
- Shared memory: the kernel is asked once, then the two processes read and write ordinary memory — and the coordination is entirely theirs.
- Faster, and back to `count++`: the race is the same one threads have.



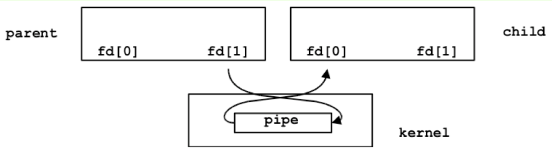
The pipe

`pipe(fd)` returns two descriptors on one kernel buffer: `fd[0]` to read, `fd[1]` to write, and after a `fork` both processes hold both.



It is one way and it is between relatives: the only way in is to inherit a descriptor that already exists.

Closing the ends you do not use




```c
int fd[2];
pipe(fd);                           /* error check omitted on the slide */
if (fork() == 0) {
    close(fd[1]);                   /* the reader gives up the write end */
    n = read(fd[0], buf, sizeof buf);
    write(STDOUT_FILENO, buf, n);
    _exit(0);
}
close(fd[0]);                       /* the writer gives up the read end */
write(fd[1], "hello\n", 6);
close(fd[1]);                       /* now, and only now, the child sees end of file */
```

The read that never returns zero

The parent writes one line, closes `fd[1]` and waits. The child loops on `read` and prints. It never ends. Why?

Because the child did not close its own `fd[1]`. One write end is still open — its own — so the pipe is never at end of file, `read` blocks, and the parent waits for a child that waits for itself.

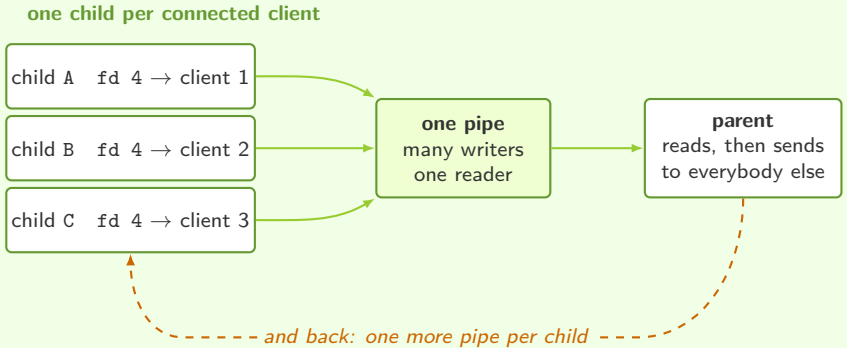
- End of file on a pipe means no writer left, and the count includes the reader's own copy.
- It is the same rule as the connected socket: the object dies when its last descriptor goes, whoever holds it.
- The mirror case is loud instead of silent: write into a pipe whose reader is gone and the process gets `SIGPIPE`.

Reading and writing a pipe

	the other end is open	**the other end is closed**
`read`, data available	returns what is there	returns what is there
`read`, pipe empty	blocks	returns 0
`write`, room in buffer	copies and returns	SIGPIPE, `EPIPE`
`write`, buffer full	blocks	SIGPIPE, `EPIPE`

- A `write` of up to `PIPE_BUF` bytes — 4096 on Linux — is atomic: several writers cannot interleave inside it.
- That is the threshold for atomicity, not the size of the pipe: the buffer itself holds 64 KB, and a writer blocks when it is full.

Children reporting to the parent



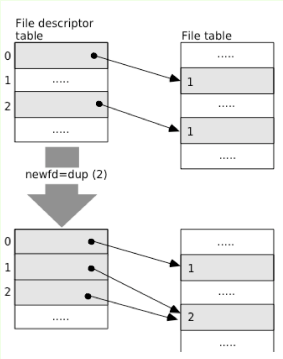
Every line crosses the kernel twice and is copied once per recipient, and the parent must stay in a loop of its own. With threads the same thing is one `for` over an array.

One pipe upwards is enough — writes under `PIPE_BUF` do not interleave — and coming back down is the expensive half.

dup and dup2

```
1 int dup(int oldfd);
2 int dup2(int oldfd, int newfd);
```

- Both make a second descriptor on the same object: same offset, same connection, one shared reference count.
- `dup` takes the lowest free number, `dup2` takes the number you name — closing it first if it was open.
- Which is the point: you can choose to land on 0 or 1.

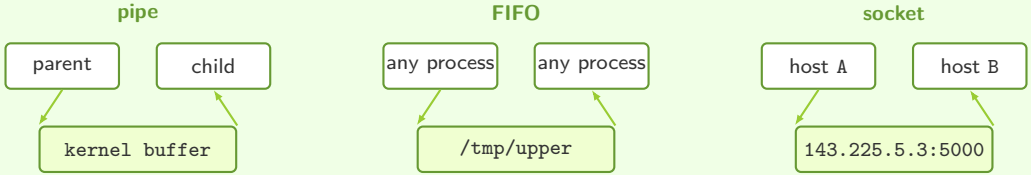


A program that does not know it is on a network

```
if (fork() == 0) {
    close(listen_fd);
    dup2(fd, STDIN_FILENO);          /* the connection becomes standard input */
    dup2(fd, STDOUT_FILENO);         /* ... and standard output */
    if (fd > STDERR_FILENO) close(fd);   /* the two duplicates are enough */
    execl("/usr/local/bin/quoted", "quoted", (char *) NULL);
    perror("execl");                     /* reached only if exec failed */
    _exit(127);
}
close(fd);
```

The program on the other side of `execl` reads with `scanf` and writes with `printf` and never learns where its input comes from. That is how a service started by `inetd` works, and how a web server runs a CGI program.

Kinship, a name, an address



Three ways for two processes to find each other, and at every step they need to have less in common.

A FIFO server and its client

```c
/* server: creates the name, then waits on it */
mkfifo("/tmp/upper", 0666);                 /* already there is not an error */
int fd = open("/tmp/upper", O_RDONLY);      /* blocks until a writer opens */
while ((n = read(fd, buf, sizeof buf - 1)) > 0) {
    buf[n] = '\0';
    for (int i = 0; i < n; i++) buf[i] = toupper((unsigned char) buf[i]);
    printf("%s\n", buf);
}

/* client: needs nothing but the same name */
int fd = open("/tmp/upper", O_WRONLY);      /* blocks until a reader opens */
write(fd, argv[1], strlen(argv[1]));
close(fd);
```

Two programs, one waiting and one asking, and no network: the name in the file system is doing what an address and a port do later, and `unlink` is the only thing that takes it away.

Shared memory across a fork

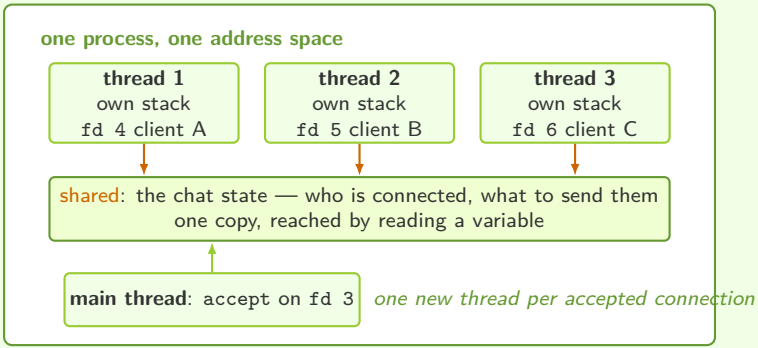
```c
unsigned long *served = mmap(NULL, sizeof *served,
                             PROT_READ | PROT_WRITE,
                             MAP_SHARED | MAP_ANONYMOUS, -1, 0);
if (served == MAP_FAILED) { perror("mmap"); exit(EXIT_FAILURE); }
*served = 0;

/* every child forked from here on sees this same counter */
```

- `MAP_ANONYMOUS` means no file behind it: a page of memory the children inherit as shared instead of copying.
- `MAP_PRIVATE` in its place gives back exactly what `fork` already gave you — a copy each.
- And `++*served` from two processes is the race we already know. A mutex there must live in this page and be created with `PTHREAD_PROCESS_SHARED`.

One thread per client

Same server, same calls, one address space: the descriptors are in one table and the state of the chat is a variable.



The accept loop that starts a thread

```c
int main(void) {
    int listen_fd = listening_socket(5000);
    signal(SIGPIPE, SIG_IGN);
    for (int i = 0; i < MAXC; i++) clients[i] = -1;

    for (;;) {
        int *fd = malloc(sizeof *fd);          /* the thread's own argument */
        if (fd == NULL) { sleep(1); continue; }
        *fd = accept(listen_fd, NULL, NULL);
        if (*fd < 0) { if (errno != EINTR) perror("accept"); free(fd); continue; }

        pthread_t tid;
        int err = pthread_create(&tid, NULL, worker, fd);
        if (err) { fprintf(stderr, "pthread_create: %s\n", strerror(err));
                   close(*fd); free(fd); continue; }
        pthread_detach(tid);                   /* nobody will ever join it */
    }
}
```

The state the threads share

```c
#define MAXC 64
static int clients[MAXC];                   /* connected sockets, -1 = free slot */
static pthread_mutex_t roster = PTHREAD_MUTEX_INITIALIZER;

static int add_client(int fd) {
    int slot = -1;
    pthread_mutex_lock(&roster);
    for (int i = 0; i < MAXC; i++)
        if (clients[i] == -1) { clients[i] = fd; slot = i; break; }
    pthread_mutex_unlock(&roster);
    return slot;                            /* -1: the chat is full, refuse */
}

static void del_client(int slot) {
    pthread_mutex_lock(&roster); clients[slot] = -1; pthread_mutex_unlock(&roster);
}
```

The worker: register, serve, deregister

```c
static void *worker(void *arg) {
    int fd = *(int *) arg;
    free(arg);                                  /* the argument was ours: done with it */

    int slot = add_client(fd);
    if (slot < 0) { write(fd, "chat is full\n", 13); close(fd); return NULL; }

    char buf[256];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0)
        broadcast(slot, buf, n);                /* everybody except me */

    del_client(slot);
    close(fd);
    return NULL;
}
```

Broadcasting to everybody else

```c
static void broadcast(int from, const char *msg, size_t len) {
    pthread_mutex_lock(&roster);
    for (int i = 0; i < MAXC; i++)
        if (clients[i] != -1 && i != from)
            write(clients[i], msg, len);   /* EPIPE: that client's own thread will see it */
    pthread_mutex_unlock(&roster);
}
```

- This is the entire difference from the multiprocess version: a `for` over a variable that every thread can see.
- The mutex is not optional — another thread may be adding itself to `clients` while this loop reads it.
- And it is held across a `write`, which is the one thing we said never to do.

The slow client that freezes the chat

One client stops reading — its window fills up. Its socket buffer fills up too. What happens to the other nineteen?

The `write` to that client blocks, with the roster mutex held. Every other thread stops at its next `broadcast`: one silent client has stopped the whole chat.

- Releasing the lock before writing does not fix it: between the copy of the list and the `write`, that slot can be closed and its number reused by a new client.
- So the write goes to the wrong connection — rarely, and never while you are watching.
- Neither horn is the answer. What a real server does is never block in a worker: each client gets an outgoing queue, and somebody drains it when the socket is ready.

Processes or threads, for a server

	one process per client	**one thread per client**
shared state	the kernel has to carry it	a variable
a client's crash	kills its own child	kills the server
a leaked descriptor	goes away with the child	stays for good
cost of starting	an address space	a stack
`close` on a socket	ends only that copy	ends it for everybody
debugging	one flow, many processes	one process, many flows

Processes pay to be isolated; threads pay for sharing. A chat wants sharing, a server that runs untrusted code wants isolation.

One worker dereferences a null pointer

A bug in the code that serves one client. What does it cost?

With processes: that client's connection dies, the parent reaps a child that ended on a signal, and nineteen conversations continue. With threads: the process gets `SIGSEGV`, and every connection dies with it.

- The same asymmetry covers a leaked buffer, a corrupted heap and a descriptor nobody closed: with processes it all ends at `_exit`.
- Which is why the choice is still made both ways: Apache ships a process model and a thread model, and the process one is chosen when the code is not trusted.
- Threads make one bug a service outage. That is the price of the `for` loop on the previous frames.

What one-per-client costs

- Both versions pay per connection, at connection time: an address space or a stack, created and destroyed for a client that may say one word.
- Both keep that cost for as long as the client stays, whether it is talking or not — and a chat client is silent almost always.
- Ten thousand idle connections means ten thousand stacks and ten thousand kernel objects, to do nothing.
- And a burst of short connections spends more time creating and tearing down than serving.

Both limits point the same way: stop making one flow of control per client. One that watches many connections and notices which are ready, and a fixed set of workers that a queue keeps busy.

Summary

- A concurrent server is the same service in a different loop: `fork` or `pthread_create` right after `accept`, and the parent or the main thread back in `accept` at once.
- With processes the price is paid up front: children see nothing of each other, so anything shared crosses the kernel — a pipe, or a page of `mmap` with a lock in it.
- A pipe joins relatives, a FIFO joins anybody with the name, a socket joins anybody with the address: the same client-server shape three times.
- `dup2` puts a connection on descriptor 0 and 1, and a program that knows nothing about networks becomes a network service.
- With threads sharing is free and safety is not: the roster needs a mutex, and holding it across a `write` hands the whole chat to the slowest client.
- Isolation against sharing, and one cost neither of them avoids: one flow of control per connection.

Iteration 2: a multiprocess chat

Where you restart from

Your sequential server: correct, and unusable as soon as a second client connects — the one you left waiting with `ss -tn`.

- What changes: the accept loop only. `fork` after `accept`, the two `close`, `_exit` in the child, and a `SIGCHLD` handler installed before the first client.
- What does not: `serve()` is untouched. If you find yourself editing it, you are solving the wrong problem.

```
$ for i in $(seq 1 20); do sleep 60 | nc 127.0.0.1 5000 & done
$ ps --ppid $(pgrep -o chatd) | wc -l      and then kill one client
```

Twenty children and no zombies: prove both. And what does the counter say to the twentieth client?

Iteration 3: the same chat with threads

Where you restart from

Iteration 2, which serves everybody and tells nobody: a line typed by one client reaches only the child that read it.

- What changes: `pthread_create` and `pthread_detach` in the loop, one `malloc` per connection for the argument, and the worker of these frames.
- What is new: the roster and its mutex, so that a line from one client goes to all the others: this is the first version that is a chat.
- Then measure: memory per connected client, with 50 clients, on both versions.

```
$ ps -o pid,nlwp,rss,cmd -p $(pgrep -o chatd)    then the same with iteration 2
```

Which version costs more per client, and which one would you still choose if `serve()` ran somebody else's code?